SplitSmart

Scan a receipt, assign items, split fairly, settle with the fewest payments. Web Brawl GC 2026 entry. React 18 + TypeScript static PWA, no backend; receipt images never leave the device.

Repository: github.com/Prayag-Mohanty/split-smart · **Live:** prayag-mohanty.github.io/split-smart

Summary. SplitSmart implements all three tiers, fully on-device and live on GitHub Pages. Beyond the brief: several payers per bill, a WhatsApp-ready bill image with a UPI QR, drag-and-drop item assignment, an animated graph that collapses IOUs into the minimum payments (exact bitmask DP, not greedy), a simplify-debts toggle, expense search, per-group default splits, auto-rotation of sideways receipts, and missing-GST-slab detection. Models: Tesseract.js v5 (English LSTM, self-hosted) for OCR and Xenova/mobilebert-uncased-mnli via Transformers.js for categorisation, both used as-is in Web Workers. No alternative models were used.

Tier 1: core (all done)

- Groups and members with colour/emoji avatars; a dashboard with group totals.
- Upload, drag-and-drop or live camera, with preview, rotate and retake.
- Canvas preprocessing, then OCR with a staged progress bar; the UI stays responsive.
- Parser for items, qty, price, total, CGST/SGST, service, discount, round-off; ignores GSTIN, phone numbers and footers.
- Editable review table beside the image; words under 70% confidence highlighted yellow.
- Total check showing the exact difference, with one-tap fixes; manual expenses; live balances.

Tier 2: advanced (all done)

- Tap-avatar or drag-and-drop assignment; unassigned items block saving.
- Equal, exact, percentage and share splits with validation.
- Tax, charges and discounts shared in proportion to item subtotals.
- Paisa-exact rounding, lazy zero-shot categories (overridable), localStorage persistence.

Tier 3 (all done)

- UPI `upi://pay` links and QR codes; PDF/CSV export; JSON and share-link import.
- Analytics by category, member and time; recurring expenses; batch scan.
- PWA that works offline (OCR assets cached); accessible, with light and dark themes.

Models (pretrained, as-is, client-side)

- **OCR:** Tesseract.js v5, English LSTM model, served from our own origin.
- **Categories:** Xenova/mobilebert-uncased-mnli via Transformers.js, in a Web Worker, loaded lazily. No alternative models used.

Algorithms

Money	All amounts are integer paise, so there is no floating-point drift. The logic sits in pure, UI-free modules in `src/lib/`.
Rounding rule	Largest remainder: round every share down to the paisa, give leftover paise to the largest fractions, ties to the earlier member. ₹100 ÷ 3 = ₹33.34 / ₹33.33 / ₹33.33.
Proportional tax	Each tax/charge/discount line is shared by item subtotal: on ₹1,000 with ₹100 tax, ₹600 of items pays ₹60.
Debt simplification	Exact minimum number of payments: bitmask DP finds the most zero-sum subgroups (n − k payments). Greedy is not always optimal and is checked against brute force in tests.

Quality

91 unit tests (splits, rounding, tax, settlement vs brute force, parser) and 10 Playwright end-to-end tests on a phone-sized browser with real OCR, covering the ₹840 judging flow and the robustness cases. CI runs lint, formatting, tests and the build on every pull request, and deploys to GitHub Pages. The first screen is about 70 KB of gzipped JS; OCR, the model, PDF and QR load only when needed.





